CAN–Ethernet Bridge IP Core

RTL Design Document

Module Architecture, Signal Connectivity,
Testbench Structure, and Design Rationale

IIITDM Chennai · Final Year Project · Rev 1.0 · May 2026

Contents

1. Purpose and Scope

This document describes the RTL implementation of the CAN–Ethernet Bridge IP core. It covers module hierarchy, port connectivity, design decisions, FSM architecture, and testbench structure.

The bridge is implemented in synthesisable Verilog and verified in QuestaSim using a UVM 1.1d testbench with 25 directed and constrained-random tests. For functional requirements and protocol definitions, see the companion Functional Specification.

2. System Architecture

2.1 Module Hierarchy

```
can_eth_bridge_top
|-- bridge_regs          Host-accessible configuration and status registers
|-- bridge_upstream      CAN -> ETH data path FSM
|   +-- can_id_filter    16-entry accept/reject CAN ID filter
|-- bridge_downstream    ETH -> CAN data path FSM
|   +-- can_tx_queue     8-deep FIFO for CAN TX buffering
|-- wb_arbiter           Dual-bus (CAN + ETH) round-robin arbiter
+-- wb_adapter_32to8     32-bit <-> 8-bit Wishbone bus width adapter
```

2.2 Top-Level Signal Flow

```
Upstream:
  can_irq_i -> upstream FSM -> CAN WB reads (via arbiter -> adapter)
            -> filter -> encap -> ETH WB writes (via arbiter)
            -> eth_tx_irq_i -> release

Downstream:
  eth_rx_irq_i -> downstream FSM -> ETH WB reads (via arbiter)
              -> validate -> decap -> queue
              -> CAN WB writes (via arbiter -> adapter)
              -> can_tx_irq_i -> free BD

Host:
  CPU <-> bridge_regs (WB slave, independent of arbiter)
```

3. Module Descriptions

3.1 can_eth_bridge_top (rtl/can_eth_bridge_top.v)

Top-level integration module. Instantiates all sub-modules, generates the timestamp counter (10-bit prescaler, 16-bit free-running counter at 20 μs resolution), and connects configuration/status wires between `bridge_regs` and the data path FSMs.

The timestamp counter is implemented at the top level rather than in a sub-module because it is shared by the upstream FSM (for tunnel-mode frame headers).

3.2 bridge_upstream (rtl/bridge_upstream.v)

8-state FSM implementing the CAN → ETH data path. Internal resources:

Register	Width	Purpose
can_id_reg	29	Captured CAN identifier
can_dlc_reg	4	Data Length Code
can_eff_reg	1	Extended Frame Format flag
can_rtr_reg	1	Remote Transmission Request flag
can_data_reg	64	CAN payload (8 bytes)
tx_frame_buf[0:15]	16×32	Encapsulated Ethernet frame buffer
tx_frame_len	16	Computed frame length in bytes
byte_cnt	5	CAN read byte counter
word_cnt	5	RAM write word counter

FSM States:

State	Code	Clocks	Operation
IDLE	0	1	Wait for can_irq_i rising edge
READ_CAN	1	13	Read frame info + ID + data from CAN regs 0x10--0x1C via arbiter
FILTER	2	1–2	Pass CAN ID through can_id_filter; if dropped → IDLE
ENCAP	3	1	Build tx_frame_buf[0:15] — Ethernet + bridge header + CAN payload
WRITE_RAM	4	16	Write 16 words to shared RAM at tx_bd_addr + 0x400
PROG_BD	5	2	Write BD status (length + READY) and pointer
WAIT_TX	6	var.	Wait for eth_tx_irq_i
RELEASE	7	1	Clear BD READY, increment counters, return to IDLE

Encapsulation logic (ENCAP state):

```
tx_frame_buf[0] = dst_mac[47:16]
tx_frame_buf[1] = {dst_mac[15:0], src_mac[47:32]}
tx_frame_buf[2] = src_mac[31:0]
tx_frame_buf[3] = {ethertype, 8'hB1, 8'hDE}  // EtherType + magic
tx_frame_buf[4] = {8'h01, can_id[28:5]}       // version + ID upper
tx_frame_buf[5] = {can_id[4:0], eff, rtr, 1'b0, dlc, flags, 16'h0}
tx_frame_buf[6] = {tunnel_seq, timestamp}      // tunnel only; GW: 0
tx_frame_buf[7] = can_data[63:32]
tx_frame_buf[8] = can_data[31:0]
tx_frame_buf[9..15] = 0                        // padding to 64 bytes
```

3.3 bridge_downstream (rtl/bridge_downstream.v)

10-state FSM implementing the ETH → CAN data path.

FSM States:

State	Code	Clocks	Operation
IDLE	0	1	Wait for `eth_rx_irq_i`
READ_BD	1	2	Read RX BD status and pointer from `rx_bd_addr`
READ_RAM	2	16	Read 16 words into `rx_frame_buf`
VALIDATE	3	1	Check EtherType, magic (`0xB1DE`), version, DLC
DECAP	4	1	Extract CAN ID, DLC, flags, payload
QUEUE_CHK	5	1+	Stall if `can_tx_queue` full
ENQUEUE	6	1	Push {ID, DLC, EFF, RTR, data} into FIFO
WRITE_CAN	7	13	Write frame to CAN regs `0x10--0x1C`, issue TX request
WAIT_CAN_TX	8	var.	Wait for `can_tx_irq_i`
FREE_BD	9	1	Release RX BD, increment counters, return to IDLE

Validation logic:

```
wire valid_gateway = (rx_ethertype == 16'hCAFE) &&
                     (rx_magic == {`BRIDGE_MAGIC_HI, `BRIDGE_MAGIC_LO}) &&
                     (rx_version == 8'h01);

wire valid_tunnel  = (rx_ethertype == 16'hCABE) &&
                     (rx_magic == {`BRIDGE_MAGIC_HI, `BRIDGE_MAGIC_LO}) &&
                     (rx_version == 8'h01);

wire frame_valid = (bridge_mode == GATEWAY) ? valid_gateway :
                   (bridge_mode == TUNNEL)  ? valid_tunnel  : 1'b0;
```

3.4 bridge_regs (rtl/bridge_regs.v)

Single-cycle Wishbone slave with 32 addressable registers. Address decode uses `wb_adr_i[7:2]` (6 bits → 64 possible slots, 32 used). Six 32-bit saturating counters with pulse-increment inputs. Configuration registers load hybrid defaults on reset.

3.5 can_id_filter (rtl/can_id_filter.v)

Purely **combinational** priority-encoded match against 16 table entries. Zero-cycle latency. Logic:

```
match = (can_id == table[0]) | ... | (can_id == table[15])

if (!filter_en)        -> filter_drop = 0  (pass all)
```

```
if (filter_mode == 0)  -> filter_drop = !match  (accept-list)
if (filter_mode == 1)  -> filter_drop = match   (reject-list)
```

3.6 can_tx_queue (rtl/can_tx_queue.v)

Synchronous FIFO. 8 entries × 78 bits per entry (29-bit ID + 4-bit DLC + flags + 64-bit data, packed). Write port used by downstream ENQUEUE state; read port by WRITE_CAN state. The `full` signal provides backpressure to QUEUE_CHK. The `depth[3:0]` output is reported in the BRIDGE_STATUS register.

3.7 wb_arbiter (rtl/wb_arbiter.v)

Manages **two independent buses** (CAN and ETH), each with two requesters (upstream and downstream). Round-robin arbitration with a `last_grant` register per bus:

```
if (req_A && req_B)
    grant = (last_grant == A) ? B : A;   // alternate
else if (req_A) grant = A;
else if (req_B) grant = B;
```

ACK signals are gated by grant to prevent cross-talk:

```
up_can_ack = can_ack_i & up_can_gnt;
dn_can_ack = can_ack_i & dn_can_gnt;
```

3.8 wb_adapter_32to8 (rtl/wb_adapter_32to8.v)

Purely combinational bus width adapter. Truncates 32-bit address/data to 8 bits for the CAN controller. Zero-extends 8-bit read data to 32 bits. No state machine, no buffering.

4. Source File Organisation

```
rtl/
|-- can_eth_bridge_top.v       Top-level (369 lines)
|-- bridge_upstream.v          Upstream FSM (~530 lines)
|-- bridge_downstream.v        Downstream FSM (~440 lines)
|-- bridge_regs.v              Register file (~300 lines)
|-- can_id_filter.v            ID filter (~100 lines)
|-- can_tx_queue.v             TX FIFO (~100 lines)
|-- wb_arbiter.v               Dual arbiter (~190 lines)
|-- wb_adapter_32to8.v         Width adapter (~50 lines)
+-- can_eth_bridge_defines.v   Constants (120 lines)
```

5. Testbench Architecture

5.1 UVM Environment

```
tb_bridge_top
|-- DUT: can_eth_bridge_top
|-- Interfaces
|   |-- wishbone_slave_if   (host_wb -- to bridge_regs)
|   |-- can_wb_if           (CAN controller model)
|   +-- mem_wb_if           (ETH memory model)
|-- bridge_env
|   |-- host_agent          (WB slave driver + monitor)
|   |-- can_wb_agent        (CAN WB responder)
|   |-- mem_wb_agent        (ETH memory responder)
|   |-- bridge_scoreboard   (data integrity checker)
|   +-- bridge_coverage     (13 covergroups)
+-- 25 test classes
```

5.2 Test Categories

Category	Count	Tests
REG	2	Register access, reset defaults
FLT	2	Accept-list, reject-list
UP	5	Basic GW, EFF, DLC sweep, tunnel, filter drop
DN	6	Basic GW, bad EtherType, bad DLC, back-pressure, bad magic, tunnel
SYS	6	Queue, arbiter, E2E up/down, full-duplex, stress
RST	1	Mid-transaction reset recovery
MODE	2	Mode switch, bridge disable
CNT	1	Counter verification
Total	**25**	

5.3 Simulation Flow

```
make comp          # vlib + vlog (RTL + TB)
make opt           # vopt with +cover=bcsf
make sim TESTNAME=upstream_basic_gw_test   # single test
make regress       # all 25 tests
make regress_report  # merge UCDB + HTML coverage report
```

6. Key Design Decisions

Decision	Rationale
Single clock domain	Avoids CDC complexity; CAN and ETH PHY clocks are handled externally by their respective controllers.
Hybrid reset defaults	Allows the bridge to operate without a CPU. Pre-configured with broadcast MAC, gateway mode, filter disabled.
Combinational filter	16-entry parallel compare costs ∼200 LUTs but provides zero-latency filtering, keeping the upstream FSM simple.
8-deep TX queue	Empirically sufficient for burst traffic; deeper queue adds area but provides no benefit since CAN bus throughput is the true bottleneck.
Magic bytes `0xB1DE`	Distinct from EtherType (`0xCAFE`/`0xCABE`) to provide independent frame validation.
Round-robin arbitration	Simplest fair scheme; prevents starvation without priority inversion.
32→8 adapter	Allows both FSMs to use uniform 32-bit WB internally; the adapter is combinational (zero area overhead).
Saturating counters	32-bit counters saturate at `0xFFFFFFFF` rather than wrapping, simplifying diagnostics.

7. Resource Estimates

7.1 Memory (Flip-Flop Based)

Resource	Size (bits)	Location
TX frame buffer	512	`bridge_upstream`
RX frame buffer	512	`bridge_downstream`
CAN TX queue	624	`can_tx_queue`
Filter table	464	`bridge_regs`
Configuration regs	640	`bridge_regs`
Total	**∼2752**	

7.2 Logic

Module	Est. LUTs	Notes
`bridge_upstream`	∼400	FSM + encapsulation muxes
`bridge_downstream`	∼450	FSM + validate + decapsulation
`bridge_regs`	∼300	Address decode + 6 counters
`wb_arbiter`	∼150	2× mux + round-robin
`can_id_filter`	∼200	16× 29-bit comparators
`can_tx_queue`	∼100	FIFO pointers + mux
`wb_adapter_32to8`	∼20	Combinational truncation

Module	Est. LUTs	Notes
Total	**∼1620**	

8. Known Limitations

Limitation	Impact	Mitigation
Single clock domain	Cannot run CAN PHY at independent frequency	CDC handled by external controllers
No DMA	CPU must configure BD addresses	Hybrid defaults allow CPU-less operation
16-entry filter	Limits filterable CAN IDs	Sufficient for typical gateway ($\leq$16 nodes)
No CRC generation	Bridge relies on ETH MAC CRC	ETH MAC appends FCS automatically
8-entry TX queue	May overflow under extreme burst	Queue full causes backpressure (no data loss)

9. Revision History

Rev	Date	Author	Changes
1.0	May 2026	FYP / IIITDM	Initial release. Covers full module hierarchy, 8+10 state FSMs, arbiter design, UVM testbench (25 tests), resource estimates.